CHAPTER 2

Whose Borrow Is It? Lifetimes in Types and Methods

In Chapter 1, every reference lived in a local variable, and the compiler could see all of its uses in one function body. Real programs put references into types: a parser that reads a borrowed text, a tokenizer that hands out slices of its input, an iterator over a collection, a view of a configuration file. A type that holds a reference carries the borrow with it, so its values are subject to the same rules as the reference itself, wherever they go.

Types with references raise a question that local variables never did. When a method of such a type returns a reference, *whose borrow is it*? The result can borrow from the value the method was called on, or from the data that value itself borrows. The two answers look almost the same in a signature, differ by a single lifetime name, and have very different consequences: one lets callers keep results while they go on using the parser, the other locks the parser until the last result is gone. This chapter is about telling the two apart, choosing on purpose, and recognizing the annotation that locks a value for the rest of its life.

In this chapter you will learn:

- why a struct that stores a reference needs a lifetime parameter, and what that parameter means for every value of the type;
- the two lifetimes inside every method of `impl<'src> Parser<'src>`, the data's and the receiver's, and how a return type picks one of them;
- why the elided return type pins the parser (E0499), and when it is exactly the right contract;
- why `slice::Iter<'a, T>::next` can return `&'a T`, how to implement `Iterator` for a borrowing struct, and why a lending iterator cannot be an `Iterator`;
- lifetimes in enums, structs with one or two lifetime parameters, and traits with a lifetime parameter;
- the `&'a mut self` trap: how one extra annotation borrows a value for the rest of its life.

2.1 A struct that holds a reference

A parser that works on a borrowed text, without copying it, needs a field that is a reference. The first attempt does not compile (`code/compile_fail/ch02_missing_lifetime.rs`):

```
pub struct Parser {
    input: &str,
    pos: usize,
}
```

```
error[E0106]: missing lifetime specifier
 --> compile_fail/ch02_missing_lifetime.rs:7:12
  |
7 |     input: &str,
  |            ^ expected named lifetime parameter
  |
help: consider introducing a named lifetime parameter
  |
6 ~ pub struct Parser<'a> {
7 ~     input: &'a str,
  |
```

E0106, "missing lifetime specifier": the type of a field must be complete, and `&str` without a lifetime is not. In a function signature the compiler can often fill in a missing lifetime for you (that is lifetime elision, Chapter 3), but in a type definition it never does; Chapter 3 explains why no default would be a good one. The `help` shows the fix: declare a lifetime parameter on the struct and use it in the field. Here is the parser we will use for the rest of the chapter, from `code/examples/ch02_parser.rs`:

```
struct Parser<'src> {
    input: &'src str,
    pos: usize,
}
```

The parameter is named `'src` after what it borrows, the source text; the name is only a convention (Chapter 14 discusses naming), and `'a` would work the same. `Parser<'src>` declares a family of types, exactly like a generic struct `Wrapper<T>`: each use of the type supplies a region for `'src`, just as each use of `Wrapper` supplies a type for `T`. `Parser<'x>` and `Parser<'y>` are different types, in the same way that `Vec<u8>` and `Vec<char>` are, although lifetimes give them a relationship that ordinary type parameters do not have: a `Parser<'long>` can be used where a `Parser<'short>` is expected. That is subtyping, and Chapter 4 explains when it applies.

What the parameter means for a value is the important part. A value of type `Parser<'src>` contains a `&'src str`, so it is only valid where that reference is valid. Chapter 1 stated the rule for references in local variables: a reference must be valid wherever it may still be used. The rule for types is the same, applied to every lifetime in the type: **wherever a value of type `Parser<'x>` may still be used, the region `'x` must contain that point**. A struct with a lifetime parameter carries a borrow, and using the struct is using the borrow. This program tries to use a parser after its text is gone (`code/compile_fail/ch02_parser_outlives_text.rs`):

```
pub fn caller() {
    let parser;
    {
        let text = String::from("lifetimes are regions");
        parser = Parser::new(&text);
    } // `text` is dropped here
    println!("{} at {}", parser.input, parser.pos);
}
```

```
error[E0597]: `text` does not live long enough
  --> compile_fail/ch02_parser_outlives_text.rs:21:30
   |
20 |         let text = String::from("lifetimes are regions");
   |             ---- binding `text` declared here
21 |         parser = Parser::new(&text);
   |                              ^^^^^ borrowed value does not live long enough
22 |     } // `text` is dropped here
   |     - `text` dropped here while still borrowed
23 |     println!("{} at {}", parser.input, parser.pos);
   |                          ------------ borrow later used here
```

The labels are those of Chapter 1's dangling caller, with one difference worth noticing: the "borrow later used here" label points at `parser.input`, a use of the parser. The borrow of `text` is stored inside `parser`, so the parser keeps it live, just as `shortlist` kept the borrow of the vector live in Chapter 1.

2.2 Two lifetimes in every method

Methods are written in an `impl` block, which must declare the lifetime parameter before it can use it, exactly as `impl<T> Wrapper<T>` declares `T`:

```
impl<'src> Parser<'src> {
    fn new(input: &'src str) -> Self {
        Parser { input, pos: 0 }
    }

    /// The result borrows from the text, so earlier results stay usable
    /// across later calls.
    fn next_word(&mut self) -> Option<&'src str> {
        let input = self.input; // a copy of the field: a `&'src str`
        let rest = &input[self.pos..];
        let trimmed = rest.trim_start();
        if trimmed.is_empty() {
            return None;
        }
        let start = self.pos + (rest.len() - trimmed.len());
        let end = trimmed
            .find(char::is_whitespace)
            .map_or(input.len(), |i| start + i);
        self.pos = end;
        Some(&input[start..end])
    }
}
```

In `impl<'src> Parser<'src>`, the first `'src` declares a lifetime parameter for the block and the second one uses it: inside the block, `Self` means `Parser<'src>` for that `'src`. `new` takes a `&'src str` and returns a `Parser<'src>`, so a parser built from a borrowed text has the region of that borrow in its type.

`next_word` is where the question of this chapter comes up. Its body copies the field into a local variable, `input`, of type `&'src str` (shared references are `Copy`), and returns a slice of it. A slice of a `&'src str` is again a `&'src str`: it points into the text, not into the parser. The return type says exactly that, `Option<&'src str>`.

There is a second lifetime in this method, which you do not see because it is elided: the lifetime of the borrow `&mut self`. Every call borrows the parser mutably for some region, and that region is a fresh lifetime, different at each call. Written out in full, the method has two lifetimes:

```
// You write:
fn next_word(&mut self) -> Option<&'src str>
fn next_word(&mut self) -> Option<&str>

// The compiler reads:
fn next_word<'p>(self: &'p mut Parser<'src>) -> Option<&'src str>
fn next_word<'p>(self: &'p mut Parser<'src>) -> Option<&'p str>
```

This listing is not compiled code: it shows how the compiler reads the two signatures, with `'p` standing for the lifetime of the receiver (Chapter 3 states the rules that produce it). The first line is our method: the result borrows from the text. The second line is what you get if you write the return type without a lifetime: the elision rules then give the result the lifetime of `&mut self`, so the result borrows from the *borrow of the parser*. Notice also an implied bound (Chapter 1): the type `&'p mut Parser<'src>` is well formed only if `'src: 'p`. The text always outlives any borrow of the parser.

The first version lets callers do what you would expect from a parser:

```
fn main() {
    let text = String::from("lifetimes are regions");
    let mut parser = Parser::new(&text);
    let first = parser.next_word();
    let second = parser.next_word(); // fine: `first` does not keep `parser` borrowed
    println!("{first:?} {second:?} / {parser}");
    println!("{:?}", all_words(&text));
}
```

```
Some("lifetimes") Some("are") / Parser@13 over "lifetimes are regions"
["lifetimes", "are", "regions"]
```

`first` and `second` are both alive at the `println!`, together with a use of `parser` itself (its `Display` implementation). Each call to `next_word` borrows the parser only for the duration of the call, because nothing that the call returns is tied to that borrow. The results borrow `text`, which is shared and outlives them all. The `Display` implementation, by the way, uses `Parser<'_>`, which means "a `Parser` with any lifetime, which I do not need to name":

```
// `'_`: "the parser's lifetime parameter, whatever it is" (Chapter 6).
impl fmt::Display for Parser<'_> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Parser@{} over {:?}", self.pos, self.input)
    }
}
```

Chapter 6 is about `'_`. Here it saves declaring `impl<'src> fmt::Display for Parser<'src>` for a block that never mentions `'src`.

Since the words borrow the text and not the parser, they can even outlive the parser:

```
fn all_words(text: &str) -> Vec<&str> {
    let mut parser = Parser::new(text);
    let mut words = Vec::new();
    while let Some(word) = parser.next_word() {
        words.push(word);
    }
    words // `parser` is dropped here; the words borrow `text`, not `parser`
}
```

The parser is a local variable of `all_words`, dropped when the function returns. The returned vector contains slices of `text`, which belongs to the caller, so returning them is fine. (The signature `fn all_words(text: &str) -> Vec<&str>` ties the result to `text` by elision: there is only one input lifetime.)

The elided version pins the parser

Now write the return type without a lifetime. The body is unchanged, character for character, and it compiles; only the signature differs (`code/compile_fail/ch02_parser_elided.rs`):

```
// Elided: the result has the lifetime of the `&mut self` borrow.
pub fn next_word(&mut self) -> Option<&str> {
```

The same caller is rejected:

```
pub fn caller() {
    let text = String::from("lifetimes are regions");
    let mut parser = Parser::new(&text);
    let first = parser.next_word();
    let second = parser.next_word(); // a second `&mut parser` while `first` is live
    println!("{first:?} {second:?}");
}
```

```
error[E0499]: cannot borrow `parser` as mutable more than once at a time
  --> compile_fail/ch02_parser_elided.rs:39:18
   |
38 |     let first = parser.next_word();
   |                 ------ first mutable borrow occurs here
39 |     let second = parser.next_word(); // a second `&mut parser` while `first` is live
   |                  ^^^^^^ second mutable borrow occurs here
40 |     println!("{first:?} {second:?}");
   |                ----- first borrow later used here
```

statement	borrow of text ('src)	1st &mut parser (returns Option<&str>)	1st &mut parser (returns Option<&'src str>)
let mut parser = Parser::new(&text);	created		
let first = parser.next_word();	live	created	created, ends
let second = parser.next_word();	live	2nd borrow: E0499	2nd borrow: fine
println!("{first:?} {second:?}");	last use	first used	

The same caller with both versions of `next_word`. When the result borrows from `&mut self`, the first mutable borrow of the parser lasts as long as `first` and conflicts with the second call; when it borrows from `'src`, each mutable borrow ends with its call.

Read the labels with the expanded signature in mind:

- `first mutable borrow occurs here`, under `parser` on line 38. A method call that takes `&mut self` borrows its receiver: `parser.next_word()` is `Parser::next_word(&mut parser)`. The compiler labels the receiver.
- `second mutable borrow occurs here`, on line 39: the second call needs another `&mut parser`.

- `first borrow later used here`, under `first` on line 40. With the elided signature, `first` has type `Option<&'p str>`, where `'p` is the region of the first `&mut parser`. It is a reference *derived from* the first mutable borrow, and Chapter 1 showed that a borrow stays live as long as anything derived from it may be used. So the first mutable borrow is live until line 40, across the second call.

The figure above puts the two versions side by side. The body compiled in both cases for the same reason: the slice it returns is a `&'src str`, and since `'src: 'p`, a `&'src str` can always be used as a `&'p str` (a reference valid for a larger region can be used for a smaller one). The elided signature does not make the body do anything different. It *promises less*: it tells callers that the result may borrow from the parser, and the callers, which see only the signature, must act accordingly. This is "the signature is the contract" again, and here the contract is weaker than the body could support.

When the elided form is right

The elided form is not a mistake in general. It is the correct contract whenever the result really does point into the value behind `self`. Consider a reader that normalizes the lines of a text (lowercase, single spaces, no blank lines) into a buffer that it owns and reuses, to avoid allocating a new `String` per line (`code/examples/ch02_lending.rs`):

```
/// Yields the non-blank lines of a text, lowercased, with single spaces.
struct Normalizer<'src> {
    lines: std::str::Lines<'src>, // borrows the text
    buffer: String,               // owned, reused for every line
}

impl<'src> Normalizer<'src> {
    fn new(text: &'src str) -> Self {
        Normalizer {
            lines: text.lines(),
            buffer: String::new(),
        }
    }

    /// The line borrows from `self.buffer`, which the next call overwrites.
    fn next_line(&mut self) -> Option<&str> {
        loop {
            let raw = self.lines.next()?;
            if raw.trim().is_empty() {
                continue;
            }
            self.buffer.clear();
            for (i, word) in raw.split_whitespace().enumerate() {
                if i > 0 {
                    self.buffer.push(' ');
                }
                self.buffer.push_str(&word.to_lowercase());
            }
            return Some(&self.buffer);
        }
    }
}
```

The field `lines` is a `std::str::Lines<'src>`, the standard library's iterator over the lines of a `&'src str`: a borrowing type, with a lifetime parameter, just like our parser. The field `buffer` is owned by the reader. `next_line` returns a reference to `buffer`, so the result really borrows from `self`, and the elided return type says so. Callers use one line at a time:

```
fn main() {
    let text = String::from("  Port =   8080\n\n HOST = Example.ORG \n");
    let mut normalizer = Normalizer::new(&text);
    let mut kept: Vec<String> = Vec::new();
    while let Some(line) = normalizer.next_line() {
        println!("{line:?}");
        kept.push(line.to_owned()); // to keep a line, copy it out of the buffer
    }
    println!("{kept:?}");
}
```

```
"port = 8080"
"host = example.org"
["port = 8080", "host = example.org"]
```

To keep a line beyond the next call, the caller copies it (`to_owned`). Holding two lines at once is rejected (`code/compile_fail/ch02_lending_keep.rs`), with the very error that the elided parser produced:

```
pub fn caller(normalizer: &mut Normalizer<'_>) {
    let first = normalizer.next_line();
    let second = normalizer.next_line(); // would overwrite what `first` shows
    println!("{first:?} {second:?}");
}
```

```
error[E0499]: cannot borrow `*normalizer` as mutable more than once at a time
  --> compile_fail/ch02_lending_keep.rs:22:18
   |
21 |     let first = normalizer.next_line();
   |                 ---------- first mutable borrow occurs here
22 |     let second = normalizer.next_line(); // would overwrite what `first` shows
   |                  ^^^^^^^^^^ second mutable borrow occurs here
23 |     println!("{first:?} {second:?}");
   |                ----- first borrow later used here
```

This time the error protects you. The second call clears the buffer and writes the next line into it; if the compiler accepted the program, `first` would show the second line's text, or point into memory that `push_str` had reallocated. The labels are the same as for the parser because the signatures have the same shape. Only you know whether the data lives in the struct or in something the struct borrows; the compiler checks that your signature tells the truth.

And it does check. Claim `'src` for a line that lives in the buffer, and the body is rejected (`code/compile_fail/ch02_lending_promise.rs`):

```
impl<'src> Normalizer<'src> {
    pub fn next_line(&mut self) -> Option<&'src str> {
        let raw = self.lines.next()?;
        self.buffer.clear();
        self.buffer.push_str(raw.trim());
        Some(&self.buffer) // borrows `self`, not the text
    }
}
```

```
error: lifetime may not live long enough
  --> compile_fail/ch02_lending_promise.rs:16:9
   |
11 | impl<'src> Normalizer<'src> {
   |      ---- lifetime `'src` defined here
12 |     pub fn next_line(&mut self) -> Option<&'src str> {
   |                       - let's call the lifetime of this reference `'1`
...
```

```
16 |         Some(&self.buffer) // borrows `self`, not the text
   |         ^^^^^^^^^^^^^^^^^^ method was supposed to return data with lifetime `'src` but
it is returning data with lifetime `'1`
```

This is the "lifetime may not live long enough" error of Chapter 1, in a method. The first label marks where `'src` is declared. The second gives the elided lifetime of `&mut self` a name so that the message can talk about it: "let's call the lifetime of this reference `'1`". Names such as `'1` and `'2` never appear in your code; they are the compiler's labels for anonymous lifetimes. The primary label says the body returns data with lifetime `'1`, the borrow of the reader, where the signature promised `'src`. The `'src` version of the parser passed this check because its data really does come from the text.

> **TIP**
>
> To choose the return type of a method, ask where the returned data lives. In something the struct *borrows* (a `&'src` field): name the struct's lifetime, `-> &'src str`. In something the struct *owns* (a `String`, a `Vec`, a buffer): let the result borrow `self`, which elision does for you, `-> &str`. Computed fresh by the method: return an owned value, `-> String`.

2.3 Iterators whose items borrow from the data

The standard library's `Iterator` trait settles the question of this chapter once and for all for every iterator. Its central method is `fn next(&mut self) -> Option<Self::Item>`, and `Item` is an associated type: one type, fixed by each implementation, the same for every call of `next`. The lifetime of the `&mut self` borrow is different at every call, so `Item` cannot mention it. Whatever an iterator yields is therefore independent of the borrow of the iterator, and this is exactly what makes `collect`, `zip`, `peekable` and a plain `let a = it.next(); let b = it.next();` possible.

Take the iterator over a slice. The documentation of `std::slice::Iter` lists its implementation as `impl<'a, T> Iterator for Iter<'a, T>`, with `type Item = &'a T` and `fn next(&mut self) -> Option<&'a T>`. Shown as code (not compiled here; the standard library generates it with a macro):

```
impl<'a, T> Iterator for Iter<'a, T> {
    type Item = &'a T;
    fn next(&mut self) -> Option<&'a T> { /* ... */ }
}
```

It is our parser's design: the struct `Iter<'a, T>` borrows a slice for `'a`, and the items borrow from the slice, for `'a`, not from the `&mut self` borrow of the iterator. So two items can be held at once (`code/examples/ch02_words_iterator.rs`):

```
fn two_at_once() {
    let ports = [8080, 8443, 9000];
    let mut iter = ports.iter(); // a `std::slice::Iter<'_, i32>`
    let first = iter.next(); // `Option<&i32>`, borrowing `ports`...
    let second = iter.next(); // ...so `iter` is free for another call
    println!("{first:?} {second:?}");
}
```

Implementing `Iterator` for a borrowing struct of your own follows the same pattern. `Words<'src>` yields the words of a borrowed text:

```
/// The words of a borrowed text.
struct Words<'src> {
    rest: &'src str,
}

impl<'src> Iterator for Words<'src> {
    type Item = &'src str; // one type for every call of `next`

    fn next(&mut self) -> Option<&'src str> {
        let trimmed = self.rest.trim_start();
        if trimmed.is_empty() {
            return None;
        }
        let end = trimmed.find(char::is_whitespace).unwrap_or(trimmed.len());
        let (word, rest) = trimmed.split_at(end);
        self.rest = rest;
        Some(word)
    }
}
```

`type Item = &'src str` names the struct's lifetime, which is available because the `impl` declares it. Because the items borrow the text, everything the standard library offers for iterators works, including keeping the items after the iterator is gone:

```
fn main() {
    two_at_once();
    let text = String::from("lifetimes are regions not durations");
    // The `Words` value is a temporary, gone at the end of the statement;
    // the words it produced borrow `text` and remain usable.
    let long_words: Vec<&str> = Words { rest: &text }.filter(|w| w.len() > 3).collect();
    println!("{long_words:?}");
}
```

```
Some(8080) Some(8443)
["lifetimes", "regions", "durations"]
```

A lending iterator is not an `Iterator`

The `Normalizer` would make a fine iterator, and `for line in normalizer` would read better than `while let`. The attempt fails (`code/compile_fail/ch02_lending_iterator.rs`):

```
impl<'src> Iterator for Normalizer<'src> {
    type Item = &str; // which lifetime? not one of the iterator's parameters

    fn next(&mut self) -> Option<&str> {
        let raw = self.lines.next()?;
        self.buffer.clear();
        self.buffer.push_str(raw.trim());
        Some(&self.buffer)
    }
}
```

```
error: associated type `Iterator::Item` is declared without lifetime parameters, so using a
borrowed type for them requires that lifetime to come from the implemented type
  --> compile_fail/ch02_lending_iterator.rs:11:17
   |
11 |     type Item = &str; // which lifetime? not one of the iterator's parameters
   |                 ^
   |
```

```
note: you can't create an `Iterator` that borrows each `Item` from itself, but you can
instead create a new type that borrows your existing type and implement `Iterator` for that
new type
  --> compile_fail/ch02_lending_iterator.rs:10:25
   |
10 | impl<'src> Iterator for Normalizer<'src> {
   |                         ^^^^^^^^^^^^^^^^
```

The error has no code, and it explains itself. `type Item = &str` needs a lifetime, and the only lifetimes available for `Item` are those of the implementation, here `'src` ("requires that lifetime to come from the implemented type"). The line lives in the buffer, so `'src` would be a lie, as `ch02_lending_promise.rs` showed; the lifetime it would need is the one of each `&mut self` borrow, and `Item` cannot name it. The note says it plainly: "you can't create an `Iterator` that borrows each `Item` from itself".

An iterator whose items borrow from the iterator is called a **lending iterator** (or streaming iterator). `Iterator` cannot express one, and that is not an oversight: the guarantee that items are independent of the iterator is what makes `collect` sound. You have three options:

- **Keep the inherent method**, `fn next_line(&mut self) -> Option<&str>`, and loop with `while let`, as `ch02_lending.rs` does. You give up `for` loops and iterator adapters.
- **Yield owned items**, so that `type Item = String`. This is the choice of the standard library's `std::io::BufRead::lines`, whose iterator yields `Result<String, std::io::Error>`, one new `String` per line.
- **Follow the note's suggestion**: keep the data somewhere else, in a value that the iterator only borrows, so that the items can borrow from that value with the iterator's lifetime parameter. `slice::Iter<'a, T>` is this design: the elements live in the slice, not in the iterator.

A lending iterator *trait*, with a `next` whose items may borrow from `&mut self`, can be written with generic associated types (GATs, stable since Rust 1.65). Chapter 10 builds one, explains the `where Self: 'a` it requires, and shows why `for` loops and adapters still do not apply.

2.4 Enums with references

An enum follows the same rule as a struct: if any variant holds a reference, the enum needs a lifetime parameter. A tokenizer for lines such as `port = 8080` can return tokens that borrow their text from the input (`code/examples/ch02_tokens.rs`):

```rust
#[derive(Debug)]
enum Token<'src> {
    Key(&'src str), // borrows from the input
    Number(u64),    // owns its data
    Equals,
}

enum LexError<'src> {
    BadWord { word: &'src str, position: usize },
}

fn tokenize<'src>(line: &'src str) -> Result<Vec<Token<'src>>, LexError<'src>> {
    let mut tokens = Vec::new();
    for (position, word) in line.split_whitespace().enumerate() {
        let token = if word == "=" {
            Token::Equals
```

```
        } else if let Ok(n) = word.parse() {
            Token::Number(n)
        } else if word.chars().all(|c| c.is_alphanumeric() || c == '_') {
            Token::Key(word)
        } else {
            return Err(LexError::BadWord { word, position });
        };
        tokens.push(token);
    }
    Ok(tokens)
}
```

```
fn main() {
    let config = String::from("port = 8080\nname = ???");
    for line in config.lines() {
        match tokenize(line) {
            Ok(tokens) => {
                let shown: Vec<String> = tokens
                    .iter()
                    .map(|token| match token {
                        Token::Key(key) => format!("key {key:?}"),
                        Token::Number(n) => format!("number {n}"),
                        Token::Equals => "equals".to_string(),
                    })
                    .collect();
                println!("{}", shown.join(", "));
            }
            Err(LexError::BadWord { word, position }) => {
                println!("bad word {word:?} at position {position}")
            }
        }
    }
    // A variant without a borrow still has the type's lifetime parameter,
    // which can be any lifetime at all, `'static` included.
    let always: Token<'static> = Token::Equals;
    println!("{always:?}");
}
```

```
key "port", equals, number 8080
bad word "???" at position 2
Equals
```

Three things are worth noticing.

The signature ties everything to the input. The return type of `tokenize` mentions `'src` twice, in `Vec<Token<'src>>` and in `LexError<'src>`: both the tokens and the error borrow from `line`, and from nothing else. The key `"port"` and the bad word `"???"` are slices of the input, not copies.

The parameter belongs to the type, not to the variant. `Token::Number(8080)` and `Token::Equals` hold no reference, but their type is still `Token<'src>`, and they are subject to `'src` like any other token: a `Vec<Token<'src>>` is a borrowing collection even if it happens to contain only numbers. The last lines of `main` show the other side of this: since `Token::Equals` borrows nothing, it can be given any lifetime at all, including `Token<'static>`.

An error that borrows is cheap and precise, and limited. `LexError<'src>` points at the offending word without allocating. The price is that the error cannot outlive the input: it cannot be returned from a function that owns the line, or stored after the input is dropped. Even with an implementation of `std::error::Error`, it cannot be converted into a `Box<dyn std::error::Error>` unless the input is

`'static`, because of a hidden `'static` bound that Chapter 8 explains; Chapter 14 discusses errors that borrow versus errors that own.

The standard library's own enums work the same way, through their type parameters: `Option<&'a str>` and `Result<&'a str, E>` are enums containing a reference, and the lifetime travels in the type argument.

2.5 How many lifetime parameters?

A struct with two reference fields can use one lifetime parameter for both, or one for each. One is usually enough, because the compiler can shorten a reference to make it fit: a `&'long str` can be used as a `&'short str`. This struct (`code/examples/ch02_two_lifetimes.rs`) stores a key borrowed from a local `String` and a value borrowed from the caller's text, which lives longer:

```
/// One lifetime for both fields: the compiler shrinks the longer borrow.
struct Entry<'a> {
    key: &'a str,
    value: &'a str,
}

fn describe(text: &str) {
    let key = String::from("host");
    // The value (from `text`, or the literal) outlives `key`; both fields
    // are used as `&'a str` for an `'a` that ends with this function.
    let entry = Entry {
        key: &key,
        value: text.lines().next().unwrap_or("unknown"),
    };
    println!("{} -> {}", entry.key, entry.value);
}
```

The compiler picks one region for `'a`, small enough for the shorter of the two borrows, and views the longer one at that region. Nothing is lost, because both fields are used only inside `describe`.

One parameter stops being enough when a method hands one of the borrows back to the caller, and the caller needs it after the *other* borrow has ended. Here is a `Finder` with one lifetime for a text and a search pattern (`code/compile_fail/ch02_one_lifetime.rs`):

```
pub struct Finder<'a> {
    text: &'a str,
    pattern: &'a str,
}

impl<'a> Finder<'a> {
    pub fn new(pattern: &'a str, text: &'a str) -> Self {
        Finder { text, pattern }
    }

    pub fn first_line_with(&self) -> Option<&'a str> {
        self.text.lines().find(|line| line.contains(self.pattern))
    }
}
```

The result of `first_line_with` is a line of `text`, but its type says `Option<&'a str>`, and `'a` is also the lifetime of `pattern`. A caller with a short-lived pattern is rejected:

```
pub fn caller() {
    let text = String::from("[server]\nport = 8080\nhost = example.org");
    let found;
    {
        let pattern = String::from("port");
        let finder = Finder::new(&pattern, &text);
        found = finder.first_line_with();
    } // `pattern` is dropped here
    println!("{found:?}");
}
```

```
error[E0597]: `pattern` does not live long enough
  --> compile_fail/ch02_one_lifetime.rs:28:34
   |
27 |         let pattern = String::from("port");
   |             ------- binding `pattern` declared here
28 |         let finder = Finder::new(&pattern, &text);
   |                                  ^^^^^^^^ borrowed value does not live long enough
29 |         found = finder.first_line_with();
30 |     } // `pattern` is dropped here
   |     - `pattern` dropped here while still borrowed
31 |     println!("{found:?}");
   |                -----
   |                borrow later used here
```

Follow the constraints. `found` is used on line 31, so `'a` must reach line 31. `&pattern` is passed as a `&'a str` on line 28, so the borrow of `pattern` must last for all of `'a`. But `pattern` is dropped on line 30. The single parameter has welded the two borrows together. Give each borrow its own parameter, and let the result name only the one it comes from:

```
/// Two lifetimes: the result names `'t` only, so the pattern may die first.
struct Finder<'t, 'p> {
    text: &'t str,
    pattern: &'p str,
}

impl<'t, 'p> Finder<'t, 'p> {
    fn new(pattern: &'p str, text: &'t str) -> Self {
        Finder { text, pattern }
    }

    fn first_line_with(&self) -> Option<&'t str> {
        self.text.lines().find(|line| line.contains(self.pattern))
    }
}

fn main() {
    describe("example.org");
    let text = String::from("[server]\nport = 8080\nhost = example.org");
    let found;
    {
        let pattern = String::from("port");
        let finder = Finder::new(&pattern, &text);
        found = finder.first_line_with();
    } // `pattern` is dropped here
    println!("{found:?}"); // fine: `found` borrows `text` only
}
```

```
host -> example.org
Some("port = 8080")
```

`first_line_with` returns `Option<&'t str>`, and nothing ties `'t` to `'p`, so the pattern may die first. This is the same fix as `first<'a, 'b>` in Chapter 1, moved from a function to a type.

The guideline that follows: **start with one lifetime parameter, and add a second when a method returns one of the borrows and callers need it to outlive the other.** Every extra parameter has a cost. Each mention of the type gets longer (`Finder<'t, 'p>`), every `impl` must declare both, and relations between them sometimes have to be written out. Mutable references add a twist that Chapter 4 explains: the compiler cannot shorten a lifetime that appears *behind* a `&mut` (in `&'a mut Vec<&'a str>`, the inner `'a` is fixed), so a single lifetime used both for a mutable reference and for what it points to is far more constraining than it looks. The last section of this chapter shows the extreme case. Chapter 14 comes back to the design question.

2.6 Traits with a lifetime parameter

A trait can have a lifetime parameter too. This is how a trait says that its methods return data borrowed from *something other than* `self`, for a region that is part of the trait. Here is a trait for sources of pieces of text (`code/examples/ch02_source_trait.rs`):

```
/// Something that hands out pieces of text borrowed for `'a`.
trait Source<'a> {
    fn next_piece(&mut self) -> Option<&'a str>;
}
```

`next_piece` borrows the source mutably, for the duration of a call, and returns a `&'a str` whose lifetime comes from the trait, not from `&mut self`. It is the `'src` version of our parser, as a trait. An implementation for a borrowing struct ties the trait's parameter to the struct's:

```
/// The comma-separated fields of a borrowed line.
struct Fields<'src> {
    rest: Option<&'src str>,
}

impl<'src> Source<'src> for Fields<'src> {
    fn next_piece(&mut self) -> Option<&'src str> {
        let rest = self.rest?;
        match rest.split_once(',') {
            Some((head, tail)) => {
                self.rest = Some(tail);
                Some(head.trim())
            }
            None => {
                self.rest = None;
                Some(rest.trim())
            }
        }
    }
}
```

`impl<'src> Source<'src> for Fields<'src>` reads: for every region `'src`, a `Fields<'src>` is a source of `&'src str` pieces. A type that hands out string literals can do better, and implement the trait for every lifetime at once:

```
/// Hands out string literals, so it is a `Source<'a>` for every `'a`.
struct Defaults {
    left: usize,
}

impl<'a> Source<'a> for Defaults {
```

```
    fn next_piece(&mut self) -> Option<&'a str> {
        if self.left == 0 {
            return None;
        }
        self.left -= 1;
        Some("default")
    }
}
```

The literal `"default"` has type `&'static str`, and a reference valid for the whole program can be used as a `&'a str` for any `'a`; so the implementation holds for all `'a` at once. Generic code can now collect pieces while it goes on calling `next_piece`:

```
/// The pieces borrow from the source's data, not from `source` itself.
fn collect_all<'a, S: Source<'a>>(source: &mut S) -> Vec<&'a str> {
    let mut pieces = Vec::new();
    while let Some(piece) = source.next_piece() {
        pieces.push(piece);
    }
    pieces
}

fn main() {
    let line = String::from("name, port, host");
    let mut fields = Fields { rest: Some(&line) };
    let pieces = collect_all(&mut fields);
    let mut defaults = Defaults { left: 2 };
    let more = collect_all(&mut defaults);
    println!("{pieces:?} {more:?}");
}
```

```
["name", "port", "host"] ["default", "default"]
```

`collect_all` is generic over both the lifetime `'a` and the source type `S`. Its result, `Vec<&'a str>`, does not mention the lifetime of `source`: the borrow of the source ends with the function, and the pieces survive it.

Without the lifetime parameter, the trait can only promise pieces that borrow from `&mut self`, and the same function is rejected (`code/compile_fail/ch02_source_elided.rs`):

```
pub trait Source {
    fn next_piece(&mut self) -> Option<&str>; // borrows from `&mut self`
}

pub fn collect_all<S: Source>(source: &mut S) -> Vec<&str> {
    let mut pieces = Vec::new();
    while let Some(piece) = source.next_piece() {
        pieces.push(piece);
    }
    pieces
}
```

```
error[E0499]: cannot borrow `*source` as mutable more than once at a time
  --> compile_fail/ch02_source_elided.rs:12:29
   |
10 | pub fn collect_all<S: Source>(source: &mut S) -> Vec<&str> {
   |                                       - let's call the lifetime of this reference `'1`
11 |     let mut pieces = Vec::new();
12 |     while let Some(piece) = source.next_piece() {
   |                             ^^^^^^ `*source` was mutably borrowed here in the previous
iteration of the loop
...
```

```
15 |     pieces
   |     ------ returning this value requires that `*source` is borrowed for `'1`
```

This diagnostic introduces two new labels. On line 10, the compiler names the elided lifetime of `source` `'1`. On line 15, "returning this value requires that `*source` is borrowed for `'1`": the return type `Vec<&str>` has the elided lifetime `'1` too, so every piece pushed into `pieces` must be valid for `'1`. Each piece comes from a call `source.next_piece()` that borrows `*source` (the value behind the reference); since the piece borrows from that call's borrow, the borrow must last for all of `'1`, which extends past the end of the function. The primary label, on line 12, is where two such borrows meet: "`*source` was mutably borrowed here in the previous iteration of the loop". As in Chapter 1's loop, the borrow made in one iteration is still live when the next iteration borrows again.

Traits with lifetime parameters are common in libraries that hand out borrowed data. The best-known example is serde's `Deserialize<'de>`, where `'de` is the lifetime of the data being deserialized from; it is what lets a deserialized struct borrow string slices from its input, which serde calls zero-copy deserialization (https://serde.rs/lifetimes.html). Chapter 9 shows how generic code requires such a trait "for every lifetime", with `for<'de> Deserialize<'de>`.

2.7 The `&'a mut self` trap

The last pattern of this chapter is the one most worth recognizing on sight. In `impl<'src> Parser<'src>`, the name `'src` is right there, and it is tempting to use it for the receiver as well, perhaps in an attempt to silence a lifetime error by giving everything the same name (`code/compile_fail/ch02_borrowed_forever.rs`):

```rust
// The same name for the borrow of the parser and for the text it reads.
pub fn next_word(&'src mut self) -> Option<&'src str> {
```

The method compiles, and so does a first call. The second call does not, even when the caller throws the first result away:

```rust
pub fn caller(text: &str) {
    let mut parser = Parser::new(text);
    parser.next_word(); // the result is thrown away...
    parser.next_word(); // ...and the parser is still borrowed
}
```

```
error[E0499]: cannot borrow `parser` as mutable more than once at a time
  --> compile_fail/ch02_borrowed_forever.rs:38:5
   |
37 |     parser.next_word(); // the result is thrown away...
   |     ------ first mutable borrow occurs here
38 |     parser.next_word(); // ...and the parser is still borrowed
   |     ^^^^^^
   |     |
   |     second mutable borrow occurs here
   |     first borrow later used here
```

Two labels share one span on line 38: the second call is both the "second mutable borrow" and the place where the "first borrow [is] later used". There is no variable holding the first result, so what uses the first borrow? The parser itself. Follow the signature: `&'src mut self` means

`self: &'src mut Parser<'src>`, so the first call borrows `parser` mutably *for the region* `'src`. And `'src` is the lifetime in the type of `parser`, which, by the rule at the start of this chapter, must contain every point where `parser` is still used. So the first mutable borrow covers every later use of the parser, and every later use conflicts with it. After the first call, the parser is borrowed for the rest of its life: it is **borrowed forever**.

Why can the compiler not choose a shorter region for this call? For a shared reference it could, and it does. With `&'src self`, the compiler is allowed to view a `Parser<'long>` as a `Parser<'short>` for the duration of the call, pick `'short` as small as the call needs, and the parser is free again afterwards. This program compiles (`code/examples/ch02_shared_self.rs`):

```
impl<'src> Parser<'src> {
    fn rest(&'src self) -> &'src str {
        &self.input[self.pos..]
    }

    fn advance(&mut self, bytes: usize) {
        self.pos = (self.pos + bytes).min(self.input.len());
    }
}

fn main() {
    let text = String::from("lifetimes are regions");
    let mut parser = Parser::new(&text);
    let before = parser.rest();
    println!("{before}"); // last use of `before`
    parser.advance(10); // fine: the shared borrow has ended
    println!("{}", parser.rest());
}
```

```
lifetimes are regions
are regions
```

Behind a `&mut`, that shortening is forbidden: a mutable reference could be used to store a short-lived reference into a value whose type promises a long-lived one. So the type behind `&'src mut` must be exactly the parser's own type, and the borrow inherits the parser's whole region. This property (`&mut T` is *invariant* in `T`, while `&T` is *covariant*) is the subject of Chapter 4, which explains the "borrowed forever" effect from the type system's point of view.

Even the shared version buys nothing. Its result is limited to the shortened `'src`, which is the region of the shared borrow of the parser, so it behaves exactly like an elided `&self` method: while the result is in use, the parser cannot be mutated (`code/compile_fail/ch02_shared_self_keep.rs`):

```
pub fn caller(text: &str) {
    let mut parser = Parser::new(text);
    let before = parser.rest();
    parser.advance(10);
    println!("{before}"); // `before` keeps the shared borrow of `parser` live
}
```

```
error[E0502]: cannot borrow `parser` as mutable because it is also borrowed as immutable
  --> compile_fail/ch02_shared_self_keep.rs:28:5
   |
27 |     let before = parser.rest();
   |                  ------ immutable borrow occurs here
28 |     parser.advance(10);
```

```
   |     ^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
29 |     println!("{before}"); // `before` keeps the shared borrow of `parser` live
   |                ------ immutable borrow later used here
```

> **WARNING**
>
> In `impl<'a> S<'a>`, never write `&'a mut self`. It ties the borrow of the value to the lifetime of the data inside it, and the value can never be used again after the call. The fix is to leave the receiver alone (`&mut self`) and put `'a` only where the data comes from, typically the return type.

The same shape, `&'a mut T<'a>`, also appears in function parameters and struct fields, and is one of the most common sources of confusing borrow errors in real code. Chapter 3 meets it as the consequence of a tempting but wrong elision rule, and Chapter 14 shows how to refactor it away.

2.8 Common confusions

"**`'src` is the lifetime of the parser.**" It is the region of the borrowed text. A parser can be created and dropped many times within it: `all_words` drops its parser long before the words it returned stop being usable.

"**The elided return type is what the compiler chose, so it must be right.**" Elision applies a fixed rule that looks only at the signature (Chapter 3). For a method with `&self` or `&mut self`, it picks the receiver, which is right when the result points into data the struct owns and wrong for views and cursors that point into borrowed data.

"**To fix E0499 on a method call, add `'a` to `&mut self`.**" That annotation is the trap of the last section. The fix for a method that pins its receiver is to name the struct's lifetime in the *return type* and leave `&mut self` elided.

"**A lending iterator just needs the right annotation.**" With `Iterator`, no annotation works: `Item` cannot depend on the borrow of `self`. Use an inherent method, owned items, or a GAT-based trait (Chapter 10).

"**More lifetime parameters make a struct more flexible, so more is better.**" Each parameter is something every user of the type has to reason about. One is usually enough; add the second when a result must outlive one of the borrows.

"**A struct with a lifetime parameter cannot be returned from a function.**" It can, whenever the data it borrows belongs to the caller: `Parser::new` returns a `Parser<'src>`, and `tokenize` returns tokens borrowing its argument.

"**`Token::Equals` has no lifetime.**" Its *type*, `Token<'src>`, has one. The lifetime is a property of the type, and every value of the type is subject to it.

2.9 Summary

- A struct or enum that stores a reference needs a lifetime parameter (`Parser<'src>`), which is part of its type. Wherever a value of type `Parser<'x>` may be used, `'x` must contain that point: using the struct is using the borrow.
- In `impl<'src> Parser<'src>`, every method has two lifetimes: `'src`, the region of the data the parser borrows, and the anonymous lifetime of the `&self` or `&mut self` borrow. A return type chooses between them.
- `-> Option<&'src str>` borrows from the data: results can be kept, collected, and can outlive the parser. The elided `-> Option<&str>` borrows from the receiver and keeps the parser exclusively borrowed while any result lives (E0499).
- The elided form is correct when the result points into data the struct owns, such as a reused buffer; claiming `'src` there is rejected with "lifetime may not live long enough".
- `Iterator::Item` cannot depend on the borrow of the iterator, which is why `slice::Iter<'a, T>` yields `&'a T` and why items can be collected. A lending iterator cannot implement `Iterator`; use an inherent method, owned items, or the GAT-based design of Chapter 10.
- One lifetime parameter usually suffices for several reference fields; use two when a method returns one borrow that callers need after the other has ended.
- A trait with a lifetime parameter (`trait Source<'a>`) lets methods return data that borrows from something other than `self`.
- `&'a mut self` in `impl<'a> S<'a>` borrows the value for its whole lifetime ("borrowed forever"), because `&mut` cannot shrink the type behind it (Chapter 4). Leave the receiver elided and name `'a` in the return type.

2.10 Exercises

1. Add a method `fn peek(&self) -> Option<&'src str>` to the parser of `code/examples/ch02_parser.rs` that returns the next word without advancing. Show that a peeked word can be kept across a call to `next_word`. Then elide its return type and explain the error. *Hint:* `peek` can run the body of `next_word` on a copy of `self.pos`. With the elided type, the peeked word holds a shared borrow of `parser`, and `next_word` needs a mutable one.
2. Implement `Iterator` for `Parser<'src>` itself, with `type Item = &'src str`, and rewrite `all_words` as `Parser::new(text).collect()`. Why does the implementation compile, when the one for `Normalizer` did not? *Hint:* `next` can simply call `self.next_word()`; ask where each item lives.
3. Change `Normalizer` so that it implements `Iterator<Item = String>`. What changes for the caller, and what does it cost? Compare with `std::io::BufRead::lines`. *Hint:* `kept.push(line.to_owned())` disappears, and so does the reuse of the buffer: there is now one allocation per line, even for lines the caller only prints.
4. A `Replace<'a>` struct holds a text, a pattern and a replacement, all as `&'a str`, and has a method `fn first_match(&self) -> Option<&'a str>` that returns the part of the text that matches. Which fields should get their own lifetime parameter if callers build the pattern and the replacement in

a short-lived scope and keep the match? Write both versions and a caller. *Hint:* only the field that the result borrows from needs to be separate; `Finder<'t, 'p>` is the model.

5. Implement `Source<'a>` for `std::slice::Iter<'a, String>`, so that `collect_all(&mut lines.iter())` works when `lines` is a `Vec<String>`. Which lifetime does each piece have? *Hint:* the body is `self.next().map(String::as_str)`. The pieces borrow the strings of the vector, for the `'a` of the slice iterator.
6. A colleague's lexer has `impl<'a> Lexer<'a> { fn skip(&'a mut self) { ... } }` and complains that "the lexer can only be used once". Reproduce the problem, explain it with the labels of the diagnostic, and fix it. Would `fn skip(&'a self)` have the same problem? *Hint:* compare `ch02_borrowed_forever.rs` and `ch02_shared_self.rs`; a method that skips input needs `&mut self` anyway.